Proposta de Webinar

C++ Threads sem mistério: Do bloqueio à responsividade, na prática

Resumo

Neste webinar, vamos apresentar de forma simples e prática como threads podem ser utilizadas em C++ para evitar que aplicações visuais fiquem travadas durante cálculos, processamento de dados, acesso a arquivos, comunicação e outras tarefas demoradas. A proposta foi construída especialmente para quem ainda não domina o assunto e quer entender, visualmente, o que acontece quando todo o trabalho é executado na Main Thread.

O conteúdo parte de uma situação comum: uma aplicação possui capacidade de CPU disponível, mas mesmo assim parece travada para o usuário. A partir desse cenário, serão explicados conceitos como processo, thread, Main Thread, Worker Thread e a diferença entre thread e núcleo ou processador lógico da CPU, sempre com linguagem acessível e exemplos próximos do dia a dia.

A parte prática utilizará C++Builder como ambiente de desenvolvimento, mas os conceitos apresentados são aplicáveis ao desenvolvimento C++ em geral. Serão comparadas duas versões de um mesmo laboratório de performance: uma executando cálculo e persistência diretamente na Main Thread e outra transferindo o trabalho pesado para uma Worker Thread.

Durante os testes, acompanharemos métricas como operações por segundo, uso de CPU, utilização da Main Thread, utilização da Worker Thread e tempo sem resposta da interface. O Gerenciador de Tarefas do Windows também será utilizado para relacionar o comportamento observado na aplicação com o escalonamento realizado pelo sistema operacional.

Ao final, o participante terá uma visão prática sobre quando uma Worker Thread pode ser útil, como devolver resultados para a Main Thread com TThread::Queue ou TThread::Synchronize e por que multithreading não deve ser entendido apenas como uma forma de 'deixar o programa mais rápido', mas também como uma ferramenta importante para preservar a responsividade e a experiência do usuário.

Público-Alvo

• Desenvolvedores C++ que desejam iniciar no estudo de threads e multithreading
• Desenvolvedores C++Builder e usuários de RAD Studio
• Desenvolvedores de aplicações desktop com interfaces VCL ou FMX
• Profissionais de automação, telemetria, IoT e integração de sistemas
• Engenheiros e estudantes de Engenharia, Automação, Computação e Tecnologia
• Profissionais que trabalham com processamento contínuo, arquivos, banco de dados, rede ou comunicação com dispositivos
• Desenvolvedores interessados em entender melhor o relacionamento entre software, threads e recursos da CPU

Key Points

• O que é uma thread e como ela se relaciona com um processo
• Main Thread x Worker Thread
• Thread não é núcleo da CPU
• Como o Windows Scheduler distribui threads entre processadores lógicos
• Por que uma aplicação pode travar mesmo com capacidade de CPU disponível
• O que acontece quando interface, cálculo e persistência disputam a mesma Main Thread
• Comparação prática entre uma aplicação bloqueante e uma aplicação com Worker Thread
• Medição de responsividade e tempo sem resposta da interface
• Comparação entre CPU do sistema, CPU do processo, Main Thread e Worker Thread
• Uso do Gerenciador de Tarefas para visualizar os processadores lógicos
• Quando uma tarefa é uma boa candidata a Worker Thread
• Retorno seguro para a interface com TThread::Queue e TThread::Synchronize
• Aplicações de threads em telemetria, ERP, sistemas financeiros, hospitalares, e-commerce e software industrial
• Cuidados básicos ao trabalhar com múltiplas linhas de execução

Benefícios

• Entender threads sem depender de uma abordagem excessivamente teórica

• Compreender por que a Main Thread é crítica em aplicações visuais
• Aprender a identificar tarefas que podem bloquear a experiência do usuário
• Visualizar a diferença entre responsividade e aumento real de velocidade de processamento
• Entender que uma Worker Thread não elimina o trabalho, apenas muda onde ele é executado
• Aprender a observar Main Thread e Worker Thread separadamente durante um teste de performance
• Conhecer uma forma segura de atualizar componentes visuais após o processamento em background
• Relacionar o comportamento da aplicação com o uso real da CPU e o escalonamento do Windows
• Aplicar o conceito em projetos de telemetria, engenharia, automação, comunicação, banco de dados e processamento contínuo
• Criar uma base conceitual para estudos futuros sobre sincronização, dados compartilhados e múltiplas Worker Threads

Tecnologias

• C++
• RAD Studio 13.2 Florence
• C++Builder
• VCL
• TThread
• TThread::Queue
• TThread::Synchronize
• APIs do Windows para medição de CPU e threads
• Gerenciador de Tarefas do Windows
• Métricas de performance e responsividade
• Persistência em arquivo CSV
• Biblioteca padrão do C++ para temporização e cálculos

Dinâmica do Webinar

O webinar começará com um problema simples e reconhecível: uma aplicação visual que parece travada enquanto executa um trabalho pesado. A partir desse comportamento, os conceitos serão construídos gradualmente, sem pressupor conhecimento prévio sobre threads.

Primeiro, será explicado o que é uma thread, a diferença entre Main Thread e Worker Thread e por que uma thread não deve ser confundida com um núcleo da CPU. Uma analogia visual com uma confeitaria será utilizada para mostrar a evolução de um único fluxo de trabalho para a separação de responsabilidades.

Na primeira demonstração, um projeto C++ executará cálculo, persistência e atualização visual na Main Thread. Será possível observar a interface deixando de responder, medir o maior tempo sem resposta e comparar o resultado com a utilização de CPU exibida pelo próprio programa e pelo Gerenciador de Tarefas.

Em seguida, o mesmo cenário será executado em uma segunda aplicação, agora utilizando uma Worker Thread. A carga matemática continuará existindo, porém será retirada da Main Thread. A comparação permitirá observar que a taxa de processamento pode permanecer semelhante enquanto a experiência do usuário muda significativamente.

Também será mostrado no código o fluxo Main Thread -> Worker Thread -> Main Thread: o evento de clique inicia o processamento, a classe derivada de TThread executa o trabalho pesado e TThread::Queue devolve os resultados para a Main Thread, onde labels, gráficos e demais componentes VCL podem ser atualizados com segurança.

• Problema inicial: aplicação com CPU disponível, mas interface travada
• O que é uma thread
• Thread x núcleo ou processador lógico
• Main Thread x Worker Thread
• Analogia visual para separação de responsabilidades
• Teste 1: processamento pesado executado na Main Thread
• Análise de CPU e responsividade no Gerenciador de Tarefas
• Teste 2: o mesmo processamento executado em Worker Thread
• Comparação dos resultados e do tempo sem resposta
• Leitura do código: criação da Worker, Execute(), Queue e atualização da interface
• Quando usar uma Worker Thread
• Cuidados básicos e atualização segura da interface
• Exemplos de uso em aplicações reais

• Conclusões e perguntas

A proposta é mostrar que threads não são um recurso reservado a aplicações extremamente complexas. Em muitos projetos C++, separar corretamente a interface do trabalho pesado já produz uma mudança significativa na responsividade e ajuda a criar aplicações mais previsíveis, escaláveis e agradáveis de utilizar.

Objetivo

Demonstrar, de forma prática e acessível, como threads podem ser utilizadas em C++ para separar tarefas pesadas da Main Thread, mantendo aplicações visuais responsivas enquanto cálculos, persistência ou outros processamentos continuam sendo executados.

O estudo utilizará dois projetos equivalentes para comparar diretamente o comportamento antes e depois da criação de uma Worker Thread. A demonstração permitirá observar métricas reais de performance, uso da CPU, atividade da Main Thread, atividade da Worker Thread e tempo de resposta da interface.

Ao final, o participante deverá ser capaz de reconhecer situações em que uma Worker Thread pode ser útil, entender o fluxo entre Main Thread e Worker Thread e compreender o papel de TThread::Queue e TThread::Synchronize na comunicação segura com a interface. Mais do que buscar velocidade, o objetivo é mostrar como organizar melhor a execução para preservar a experiência do usuário.